

Programas como demostraciones: la interpretación lógica de la programación funcional en Rocq

1st Gael Emiliano Arreguín Salgado
Ciencias de la Computación
Facultad de Ciencias, UNAM
Ciudad de México, México
emiliano.arreguin@ciencias.unam.mx

2nd López Pérez Mariana
Ciencias de la Computación
Facultad de Ciencias
Ciudad de México, México
marlop@ciencias.unam.mx

3rd Navarro Fierro Michelle Alanis
Ciencias de la Computación
Facultad de Ciencias
Ciudad de México, México
michellenavf0@ciencias.unam.mx

Resumen—Este trabajo analiza por qué la programación funcional en Rocq puede interpretarse como parte de un sistema lógico formal, más allá de considerarse una técnica para escribir programas. La tesis principal es que Rocq integra programas, tipos, proposiciones y pruebas dentro de un mismo marco basado en el Cálculo de Construcciones Inductivas. A partir de la correspondencia de Curry–Howard, un tipo puede leerse como una especificación lógica y un término puede leerse como un programa o como una prueba, según el universo en el que se encuentre. Sostenemos esta tesis examinando el juicio de tipo, la relación entre proposiciones y tipos, el papel de las definiciones inductivas, los objetos de prueba (*proof objects*) y un caso de estudio donde una semántica funcional se relaciona formalmente con un juicio inductivo de evaluación. Demostrando que Rocq no sólo permite ejecutar funciones, sino también formular y verificar propiedades sobre ellas dentro del propio lenguaje.

I. INTRODUCCIÓN

La relación entre programación y demostración es un aspecto fundamental de la teoría de tipos y de la verificación formal. En un lenguaje funcional convencional, una función suele ser vista como un conjunto de reglas computacionales que reciben datos de entrada, aplican reglas de evaluación y producen un resultado; a pesar de que el tipo de una función proporciona información sobre la salida, generalmente no se interpreta como una prueba lógica. Rocq altera esta lectura debido a la naturaleza funcional de su programación, integrando programas, proposiciones, pruebas y tipos dentro de un mismo sistema formal.

La pregunta que guía nuestro trabajo es la siguiente: *¿cómo se interpreta la programación funcional en Rocq como un sistema lógico y no sólo como un lenguaje de programación?* La respuesta que se defenderá es que Rocq soporta esta interpretación, ya que apela al Cálculo de Construcciones Inductivas (CIC), un marco en el que los tipos actúan como especificaciones y los términos como programas o pruebas. Esta idea se respalda en la correspondencia de Curry–Howard, la cual indica la relación estructural entre proposiciones y tipos, y entre pruebas y términos [1]–[3].

El punto técnico que organiza nuestro análisis es el juicio de tipo

$$\Gamma \vdash t : A \quad (1)$$

En una lectura computacional, el juicio afirma que el término t es un programa de tipo A bajo el contexto Γ . En una lectura

lógica, cuando A refiere a una proposición, el mismo juicio afirma que t es una prueba de A bajo las hipótesis contenidas en Γ . Esta doble lectura es una consecuencia del modo en que Rocq organiza funciones, proposiciones y demostraciones dentro de su sistema de tipos [4]–[6].

El objetivo del trabajo no es presentar una historia completa del cálculo lambda, ni desarrollar todos los detalles teóricos del CIC y conceptos muy específicos. Ese alcance sería demasiado amplio para los propósitos de nuestro trabajo. En cambio se seleccionan los elementos estrictamente necesarios para explicar la interpretación lógica de la programación funcional en Rocq, es decir, la correspondencia de Curry–Howard, el juicio de tipo, los universos `Prop` y `Type`, las definiciones inductivas, los objetos de prueba y un ejemplo formal que conecta una función con un juicio semántico inductivo.

La relevancia de esta pregunta está en que Rocq ocupa una posición intermedia entre lenguaje de programación funcional y asistente de pruebas. Por un lado nos permite definir funciones mediante recursión estructural, apareamiento de patrones y tipos algebraicos. Por otro lado, permite expresar proposiciones, construir pruebas y verificar que dichas pruebas sean términos bien tipados. Por ello programar en Rocq no se reduce a obtener código ejecutable, ya que también puede significar construir objetos formales cuyo significado lógico es verificado por el núcleo del sistema [7], [8].

Para desarrollar esta tesis, primero se explica la relación entre programas, tipos y pruebas a partir de la correspondencia Curry–Howard. Después se presenta el papel del CIC necesario para comprender a Rocq como marco lógico, es decir; universos, productos dependientes y definiciones inductivas. Luego se analiza la noción de *proof object*, que permite entender las pruebas como términos y no sólo como *scripts* de tácticas. Posteriormente se estudia un caso concreto; una semántica de expresiones aritméticas definida tanto como función como relación inductiva, junto con un teorema que prueba su equivalencia. Se compara Rocq con la programación funcional convencional y se concluye en qué sentido sus programas pueden interpretarse como construcciones lógicas.

II. PROGRAMAS, TIPOS Y PRUEBAS

La interpretación lógica de la programación funcional en Rocq se fundamenta en la correspondencia de Curry–Howard.

Esta correspondencia establece una relación estructural entre lógica y teoría de tipos; que las proposiciones pueden verse como tipos, y las pruebas como términos que habitan esos tipos [1], [4], [6]. Para demostrar una proposición no solo consiste en escribir una explicación informal, sino en construir un objeto formal que el sistema pueda verificar.

La idea puede resumirse de la siguiente manera:

Lógica	Teoría de tipos
Proposición	Tipo
Prueba	Término
$P \rightarrow Q$	$P \rightarrow Q$
$\forall x : A, P(x)$	$\Pi x : A, P(x)$

(2)

No se debe interpretar la tabla como una simple analogía. En Rocq, esta correspondencia tiene consecuencias concretas; que una proposición se representa como un tipo en `Prop` y probarla equivale a construir un término de ese tipo.

El juicio principal tiene la forma:

$$\Gamma \vdash t : A \quad (3)$$

Donde Γ es un contexto de variables e hipótesis, t es el término construido y A es su tipo. Si A describe datos computacionales, entonces t puede leerse como un programa. Si A expresa una proposición, entonces t puede leerse como una prueba. Esta doble forma de verlo es la base de la respuesta a la pregunta. Rocq permite interpretar la programación funcional como parte de un sistema lógico porque programas y pruebas son verificados mediante juicios de la misma forma [2], [3], [5].

II-A. La implicación como función

Uno de los casos más claros de Curry–Howard es la implicación. En lógica, una prueba de $P \rightarrow Q$ transforma una prueba de P en una prueba de Q . En teoría de tipos, un término de tipo $P \rightarrow Q$ es una función que recibe un argumento de tipo P y produce un resultado de tipo Q . La regla de introducción de la implicación coincide con la construcción de una función:

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x : A. t : A \rightarrow B} \rightarrow\text{-INTRO}$$

La lectura computacional dice que, bajo una variable $x : A$, se construye un término $t : B$; por tanto, se obtiene una función de A en B . La lectura lógica dice que, si bajo la hipótesis A se puede demostrar B , entonces se ha demostrado $A \rightarrow B$.

La eliminación de la implicación coincide con la aplicación funcional:

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B} \rightarrow\text{-ELIM}$$

Lógicamente, esta regla es *modus ponens*: de una prueba de $A \rightarrow B$ y una prueba de A , se obtiene una prueba de B . Computacionalmente, es la aplicación de una función a un argumento. Esta coincidencia es esencial para Rocq, porque muestra que una operación ordinaria de programación funcional puede tener simultáneamente una lectura lógica [1], [4].

En Rocq, esta idea puede escribirse directamente:

Definition modus_ponens

```
(P Q : Prop) (Himp : P -> Q) (Hp : P) : Q :=
  Himp Hp.
```

El término `modus_ponens` recibe dos proposiciones, una prueba de $P \rightarrow Q$ y una prueba de P ; después aplica `Himp` a `Hp` para obtener una prueba de Q . Como programa, es una función. Como prueba, es una formalización de la regla lógica de eliminación de la implicación. El mismo fragmento de código tiene ambas lecturas porque su significado está determinado por el juicio de tipado.

II-B. Proposiciones como tipos

La diferencia entre un tipo computacional y una proposición puede verse mediante los universos de Rocq. Tipos como `nat` o `bool` describen datos; proposiciones como `True`, `False` o `forall n : nat, n = n` pertenecen a `Prop`. Sin embargo, todos se integran dentro del mismo sistema de tipos:

```
Check nat.
Check bool.
Check True.
Check False.
Check forall n : nat, n = n.
```

Esta clasificación permite que Rocq distinga entre contenido computacional y contenido lógico sin separar ambos en lenguajes distintos. Una función sobre naturales y una prueba de una proposición son objetos diferentes por su tipo, pero ambos son términos verificados por el mismo mecanismo formal [3], [5].

Por ejemplo, la identidad sobre naturales es un programa:

```
Definition id_nat : nat -> nat :=
  fun n : nat => n.
```

Su juicio de tipado puede escribirse como:

$$\vdash \text{id_nat} : \text{nat} \rightarrow \text{nat}. \quad (4)$$

En cambio, la identidad sobre una proposición es una prueba:

```
Definition id_proof : forall P : Prop, P -> P :=
  fun P : Prop => fun H : P => H.
```

Su juicio correspondiente es:

$$\vdash \text{id_proof} : \forall P : \text{Prop}, P \rightarrow P. \quad (5)$$

La estructura del término es funcional en ambos casos. La diferencia está en que `id_nat` opera sobre datos, mientras que `id_proof` opera sobre evidencia lógica. Esto ilustra con precisión el punto central: Rocq no necesita cambiar de lenguaje para pasar de programas a pruebas; cambia la interpretación del tipo que habita el término.

II-C. Demostraciones como construcción de términos

Una demostración en Rocq puede escribirse explícitamente como una definición o construirse interactivamente mediante tácticas. En ambos casos, el resultado debe ser un término bien tipado. Por ejemplo:

```
Theorem id_proof_tactic :  
  forall P : Prop, P -> P.  
Proof.  
  intros P H.  
  exact H.  
Qed.
```

Aunque aquí la prueba se escribió con tácticas, su contenido lógico es el mismo que en la definición anterior: dado un tipo proposicional P y una prueba $H : P$, se devuelve H . El script no es la justificación final por sí mismo; la justificación final es el objeto de prueba que Rocq construye y verifica [5], [6].

Este punto evita una confusión común. Las tácticas facilitan la construcción de pruebas, pero no sustituyen al sistema lógico. Una prueba aceptada por Rocq debe terminar en un término que habita el tipo declarado. Por eso, la pregunta sobre Rocq no debe responderse diciendo solamente que “Rocq permite escribir pruebas”, sino explicando que esas pruebas son objetos internos del sistema de tipos.

II-D. Alcance de esta lectura

La lectura lógica de los programas en Rocq debe formularse con cuidado. No todo término computacional es automáticamente una prueba matemática interesante. Una función de tipo $\text{nat} \rightarrow \text{nat}$ es principalmente un programa; para obtener una afirmación lógica fuerte, es necesario formular una especificación, por ejemplo:

```
forall n : nat, f n = n
```

o una relación semántica que describa su comportamiento. Lo característico de Rocq es que esa especificación también puede expresarse como un tipo y que la prueba de la especificación también puede construirse como un término.

Así, Curry–Howard no convierte cualquier programa en una demostración útil por sí mismo. Más bien, proporciona el marco formal para que programas, especificaciones y pruebas convivan bajo el mismo juicio:

$$\Gamma \vdash t : A. \quad (6)$$

Esta es la base sobre la cual se construye el resto del trabajo: primero se entiende que los tipos pueden expresar proposiciones; después se observa que Rocq permite definir datos, relaciones y juicios inductivos; finalmente se muestra, mediante un caso de estudio, cómo una función y una especificación lógica pueden conectarse mediante una demostración formal.

III. ROCQ Y EL CIC COMO MARCO LÓGICO

Rocq se apoya en el Cálculo de Construcciones Inductivas (CIC), un sistema de tipos dependientes que permite expresar programas, proposiciones y pruebas dentro de un

mismo lenguaje formal. Para la pregunta de este trabajo, no es necesario desarrollar toda la metateoría del CIC; basta con explicar los mecanismos que hacen posible la lectura lógica de la programación funcional: los universos, los productos dependientes, las definiciones inductivas y los objetos de prueba [2], [3], [5].

El CIC permite que los tipos dependan de términos. Esta característica es más fuerte que la tipificación usual de muchos lenguajes funcionales. En un lenguaje convencional, un tipo como $\text{nat} \rightarrow \text{nat}$ clasifica funciones de naturales en naturales. En Rocq, además, pueden formularse tipos que mencionan valores concretos y expresan propiedades sobre ellos. Por ejemplo:

```
forall n : nat, n + 0 = n
```

no describe simplemente una clase de programas; expresa una proposición universal sobre todos los números naturales. Una prueba de ese enunciado debe ser un término que, dado cualquier n , produzca evidencia de la igualdad correspondiente [4], [6].

III-A. Universos y contenido lógico

Rocq organiza los objetos mediante universos. De manera simplificada, Type clasifica tipos computacionales, mientras que Prop clasifica proposiciones lógicas. Esta distinción permite separar el contenido usado para calcular del contenido usado para demostrar, sin abandonar el mismo sistema de tipos [3], [5].

En Rocq puede observarse esta clasificación con comandos como:

```
Check nat.  
Check bool.  
Check True.  
Check False.  
Check forall n : nat, n = n.
```

Los objetos nat y bool pertenecen al lado computacional usual. En cambio, True , False y $\text{forall n : nat, n = n}$ pertenecen al lado proposicional. Lo importante es que todos aparecen dentro del mismo lenguaje y son verificados mediante juicios de tipo. Esto evita pensar que Rocq tiene un lenguaje para programas y otro completamente separado para pruebas. La diferencia está en el tipo que habita cada término.

Esta clasificación también permite formular con precisión la tesis del trabajo. No se afirma que todo programa sea automáticamente una prueba profunda, sino que Rocq permite expresar especificaciones como tipos y construir términos que las habitan. Cuando el tipo pertenece a Prop , el término se interpreta como prueba; cuando el tipo describe datos o funciones computacionales, el término se interpreta como programa.

III-B. Productos dependientes y cuantificación

El producto dependiente es una construcción central del CIC. Se escribe:

$$\Pi x : A, B(x). \quad (7)$$

Computacionalmente, representa una función cuyo resultado puede tener un tipo dependiente del argumento. Lógicamente, representa cuantificación universal:

$$\forall x : A, B(x). \quad (8)$$

Una prueba de este enunciado es una función que, dado un valor arbitrario $x : A$, produce una prueba de $B(x)$ [3], [4].

Las reglas principales pueden representarse así:

$$\frac{\Gamma, x : A \vdash t : B(x)}{\Gamma \vdash \lambda x : A. t : \Pi x : A, B(x)} \text{II-INTRO}$$

$$\frac{\Gamma \vdash f : \Pi x : A, B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B(a)} \text{II-ELIM}$$

Estas reglas son relevantes porque muestran que la construcción de pruebas universales tiene forma funcional. Si se quiere demostrar una proposición de la forma `forall x : A, B x`, se construye una función que recibe un x arbitrario y devuelve una prueba de $B x$. Por tanto, una de las operaciones básicas de la programación funcional —la abstracción funcional— también es una operación lógica de introducción de cuantificadores.

Por ejemplo:

```
Definition eq_refl_nat : forall n : nat, n = n.
  fun n : nat => eq_refl.
```

Este término puede leerse como una función que recibe un natural n ; pero también como una prueba universal de que todo natural es igual a sí mismo. El juicio correspondiente es:

$$\vdash \text{eq_refl_nat} : \forall n : \text{nat}, n = n. \quad (9)$$

El ejemplo es simple, pero muestra el punto formal: la cuantificación universal y la función dependiente se verifican mediante el mismo mecanismo.

III-C. Definiciones inductivas como datos y proposiciones

El mecanismo `Inductive` es otro punto clave. En lenguajes funcionales, las definiciones inductivas suelen usarse para declarar estructuras de datos. En Rocq, además, sirven para declarar proposiciones, relaciones y juicios formales [3], [4], [9].

Como tipo de datos, los naturales pueden presentarse con dos constructores:

```
Inductive nat : Type :=
| O : nat
| S : nat -> nat.
```

Esta definición introduce valores computacionales sobre los que pueden definirse funciones por recursión estructural:

```
Fixpoint double (n : nat) : nat :=
  match n with
  | O => O
  | S n' => S (double n')
  end.
```

Pero la misma definición de `nat` genera también un principio de inducción. Informalmente, dicho principio afirma que para probar una propiedad P sobre todos los naturales basta probarla para O y probar que se preserva al pasar de n a $S n$:

$$P(O) \rightarrow (\forall n : \text{nat}, P(n) \rightarrow P(S n)) \rightarrow \forall n : \text{nat}, P(n). \quad (10)$$

En Rocq, este principio no es una regla informal externa; aparece como un término:

```
Check nat_ind.
```

Así, la estructura de datos trae consigo una herramienta lógica para razonar sobre sus habitantes. Esta es una diferencia profunda con la programación funcional convencional: definir un tipo no sólo permite construir valores, sino también obtener principios de razonamiento sobre esos valores.

El mismo mecanismo puede declarar proposiciones inductivas. Por ejemplo:

```
Inductive even : nat -> Prop :=
| ev_0 : even O
| ev_SS : forall n : nat,
  even n -> even (S (S n)).
```

Aquí `even n` no es un dato computacional ordinario, sino una proposición. Sus constructores son formas de construir pruebas. La regla `ev_0` prueba que cero es par; la regla `ev_SS` dice que si n es par, entonces $S (S n)$ también lo es.

Estas reglas pueden escribirse como juicios:

$$\frac{}{\text{even } O} \text{EV}_0 \quad \frac{\text{even } n}{\text{even } (S(S n))} \text{EV}_{SS}$$

La correspondencia con Rocq es directa: cada constructor de la definición inductiva corresponde a una regla para construir evidencia. Por ejemplo:

```
Definition ev_4 : even (S (S (S (S O)))) :=
  ev_SS (S (S O)) (ev_SS O ev_0).
```

El juicio de tipado asociado es:

$$\vdash \text{ev}_4 : \text{even } 4. \quad (11)$$

Por tanto, `ev_4` es simultáneamente un término construido con constructores y una prueba formal de que cuatro es par. Este ejemplo muestra cómo Rocq convierte reglas lógicas en objetos manipulables por el sistema de tipos.

III-D. Proof objects y tácticas

Las pruebas en Rocq pueden escribirse explícitamente como términos o construirse mediante tácticas. Sin embargo, el criterio final de validez no es el texto del script, sino el objeto de prueba que el sistema obtiene. Una prueba aceptada debe corresponder a un término bien tipado [5], [6].

La prueba anterior puede escribirse también con tácticas:

```
Theorem ev_4_tactic : even (S (S (S (S O)))) .
Proof.
  apply ev_SS.
```

```

  apply ev_SS.
  apply ev_0.
Qed.

```

Aunque esta presentación parece diferente, el resultado lógico es el mismo: Rocq construye un habitante del tipo `even 4`. Las tácticas ayudan a construir el término, pero no añaden una lógica externa al sistema. Al finalizar, el núcleo verifica que el objeto construido tenga el tipo declarado.

Este punto es central para responder la pregunta de investigación. En Rocq, una demostración no es solamente una explicación textual ni una secuencia de comandos; es un término que habita una proposición. Por eso la programación funcional y la demostración formal pueden convivir: ambas consisten en construir términos sometidos al mismo juicio de tipado.

III-E. Cómputo dentro de la prueba

Rocq también permite usar cómputo durante las demostraciones. Por ejemplo:

```

Example two_plus_two : 2 + 2 = 4.
Proof.
  reflexivity.
Qed.

```

La prueba se cierra con `reflexivity` porque ambos lados de la igualdad se reducen al mismo valor. Esto muestra que la evaluación de programas no está separada de la verificación lógica: el sistema puede calcular para comprobar que dos términos son definicionalmente iguales [4], [5].

Una demostración un poco más significativa usa inducción:

```

Theorem plus_0_r : forall n : nat, n + 0 = n.
Proof.
  induction n as [| n' IH].
  - reflexivity.
  - simpl. rewrite IH. reflexivity.
Qed.

```

Esta prueba usa el principio de inducción asociado a `nat`. En el caso base, la igualdad se resuelve por cómputo. En el caso inductivo, `simpl` reduce la expresión, `rewrite IH` usa la hipótesis inductiva y `reflexivity` cierra la igualdad resultante. La prueba demuestra que una función definida por recursión satisface una propiedad para todos los naturales.

Este ejemplo anticipa el caso de estudio de la siguiente sección. Rocq no sólo permite definir funciones como `double` o la suma sobre naturales; también permite formular propiedades universales sobre ellas y construir pruebas verificadas. Esa capacidad es lo que transforma la programación funcional en un componente de un sistema lógico.

IV. CASO DE ESTUDIO: FUNCIÓN, JUICIO SEMÁNTICO Y PRUEBA

Para mostrar de manera concreta cómo la programación funcional en Rocq puede interpretarse como parte de un sistema lógico, consideremos un lenguaje mínimo de expresiones aritméticas. El objetivo del ejemplo no es construir un lenguaje

complejo, sino exhibir una situación típica de ciencias de la computación: una semántica puede definirse como una función que calcula resultados o como un juicio inductivo que describe reglas de evaluación. Rocq permite formular ambas presentaciones y demostrar formalmente que coinciden [4], [10], [11].

Definimos primero la sintaxis de expresiones:

```

Inductive aexp : Type :=
| ANum   : nat -> aexp
| APlus  : aexp -> aexp -> aexp.

```

El constructor `ANum` representa una constante numérica, mientras que `APlus` representa la suma de dos expresiones. Esta definición es una definición inductiva de datos: introduce una estructura sobre la cual pueden escribirse funciones por recursión estructural. La semántica funcional del lenguaje puede darse mediante el siguiente intérprete:

```

Fixpoint aeval (a : aexp) : nat :=
  match a with
  | ANum n => n
  | APlus a1 a2 => aeval a1 + aeval a2
  end.

```

Desde la lectura computacional, `aeval` es un programa funcional. Su tipo es:

$$\vdash \text{aeval} : \text{aexp} \rightarrow \text{nat}. \quad (12)$$

Esto significa que `aeval` recibe una expresión aritmética y devuelve un número natural. Hasta aquí, el ejemplo podría pertenecer a un lenguaje funcional ordinario.

Sin embargo, la misma semántica también puede presentarse como un juicio lógico de evaluación. En lugar de calcular directamente un número, definimos una relación inductiva:

```

Inductive aevalR : aexp -> nat -> Prop :=
| E_ANum : forall n,
  aevalR (ANum n) n
| E_APlus : forall a1 a2 n1 n2,
  aevalR a1 n1 ->
  aevalR a2 n2 ->
  aevalR (APlus a1 a2) (n1 + n2).

```

Ahora `aevalR a n` no es un programa que devuelve un valor, sino una proposición. Su lectura es: la expresión `a` evalúa al número `n`. Los constructores `E_ANum` y `E_APlus` son reglas para construir pruebas de esa proposición. En notación semántica, pueden escribirse como:

$$\frac{}{ANum\ n \Downarrow n} E_ANUM$$

$$\frac{a_1 \Downarrow n_1 \quad a_2 \Downarrow n_2}{APlus\ a_1\ a_2 \Downarrow n_1 + n_2} E_APLUS$$

Esta presentación es propia de la semántica operacional: el significado de una expresión se justifica mediante una derivación construida con reglas de inferencia. En Rocq, dicha derivación no queda fuera del sistema; se representa como un

término que habita una proposición de la forma $\text{aevalR } a \ n$. Por tanto, el juicio semántico se convierte en un objeto formal verificable por el sistema de tipos [3], [10], [11].

La relación entre la función aeval y el juicio aevalR puede establecerse mediante una demostración. Primero probamos que toda derivación relacional es correcta respecto del intérprete funcional:

```
Theorem aevalR_sound :
  forall a n, aevalR a n -> aeval a = n.
Proof.
  intros a n H.
  induction H.
  - reflexivity.
  - simpl. rewrite IHaevalR1.
    rewrite IHaevalR2. reflexivity.
Qed.
```

Este teorema afirma que si existe una prueba de $\text{aevalR } a \ n$, entonces el programa aeval calcula precisamente n . La inducción se realiza sobre la derivación H , no directamente sobre la expresión. Esto es importante: cuando una propiedad depende de un juicio inductivo, el objeto natural de inducción es la prueba de ese juicio. Así, Rocq permite razonar sobre derivaciones semánticas como objetos matemáticos internos.

La dirección inversa muestra que toda evaluación funcional puede justificarse mediante una derivación relacional:

```
Theorem aevalR_complete :
  forall a, aevalR a (aeval a).
Proof.
  induction a as [n | a1 IH1 a2 IH2].
  - simpl. apply E_ANum.
  - simpl. apply E_APlus; assumption.
Qed.
```

Esta prueba procede por inducción estructural sobre la expresión a . En el caso numérico, se usa la regla E_ANum . En el caso de suma, las hipótesis inductivas proporcionan derivaciones para las subexpresiones, y la regla E_APlus combina esas derivaciones para construir una derivación de la expresión completa.

Finalmente, ambas direcciones pueden reunirse en una equivalencia:

```
Theorem aeval_iff_aevalR :
  forall a n, aevalR a n <-> aeval a = n.
Proof.
  intros a n. split.
  - apply aevalR_sound.
  - intros H. rewrite <- H.
    apply aevalR_complete.
Qed.
```

Este resultado es el núcleo matemático del ejemplo. La función aeval representa la lectura computacional de la semántica; la relación aevalR representa la lectura lógica como juicio inductivo; y el teorema aeval_iff_aevalR certifica que

ambas descripciones coinciden. En términos del juicio de tipado, Rocq acepta un término:

$$\vdash \text{aeval_iff_aevalR} : \forall a \ n, \text{aevalR } a \ n \leftrightarrow \text{aeval } a = n. \quad (13)$$

Este caso responde directamente la pregunta de investigación. En Rocq, una definición funcional no queda aislada como código ejecutable. Puede relacionarse con una especificación lógica, expresada como una proposición inductiva, y con una prueba formal de equivalencia. Por eso la programación funcional en Rocq tiene una lectura lógica: el programa, el juicio semántico y la demostración son objetos del mismo sistema formal.

V. DIFERENCIA CON LA PROGRAMACIÓN FUNCIONAL CONVENCIONAL

La comparación con la programación funcional convencional permite precisar el alcance de la tesis. Rocq comparte con lenguajes como OCaml, Haskell o ML varias construcciones familiares: funciones, tipos algebraicos, pattern matching, recursión estructural y funciones de orden superior. Sin embargo, en Rocq estas construcciones no aparecen sólo como recursos para escribir programas ejecutables, sino como parte de un sistema lógico donde los términos pueden expresar programas, especificaciones y pruebas [3], [7], [8].

En un lenguaje funcional convencional, un tipo como:

```
aexp -> nat
```

clasifica funciones que reciben una expresión aritmética y devuelven un número. Esta información es útil porque evita ciertos errores de uso, pero no afirma por sí misma que la función implementa correctamente una semántica. En cambio, en Rocq puede formularse una propiedad que relacione una función con una especificación lógica:

```
forall a n, aevalR a n <-> aeval a = n
```

Este tipo no sólo clasifica un programa; expresa una proposición matemática sobre el comportamiento de aeval . Una prueba de ese enunciado es un objeto formal que certifica la equivalencia entre el intérprete funcional y el juicio relacional de evaluación [4], [10], [11].

La diferencia puede expresarse mediante dos juicios:

$$\vdash \text{aeval} : \text{aexp} \rightarrow \text{nat} \quad (14)$$

y

$$\vdash \text{aeval_iff_aevalR} : \forall a \ n, \text{aevalR } a \ n \leftrightarrow \text{aeval } a = n. \quad (15)$$

El primer juicio clasifica un programa. El segundo clasifica una prueba sobre ese programa. Ambos tienen la misma forma general:

$$\Gamma \vdash t : A. \quad (16)$$

Esto muestra que Rocq no separa radicalmente el código de su justificación formal: ambos pueden aparecer como términos sometidos al mismo sistema de verificación [2], [5], [6].

En lenguajes funcionales ordinarios, la corrección de un programa suele establecerse mediante pruebas externas, razonamientos informales, pruebas unitarias o herramientas adicionales. Estas estrategias pueden ser útiles, pero no forman parte necesariamente del lenguaje mismo. En Rocq, en cambio, la especificación puede escribirse como un tipo y la demostración puede construirse dentro del sistema. Por ello, el programa no sólo se ejecuta: también puede ser relacionado formalmente con una propiedad que el núcleo verifica.

Esta diferencia se observa también en las definiciones inductivas. En un lenguaje funcional convencional, un tipo algebraico sirve principalmente para construir y analizar datos. En Rocq, una definición inductiva genera además principios de razonamiento. Por ejemplo, al definir `nat`, el sistema no sólo introduce los constructores `O` y `S`; también proporciona un principio de inducción que permite demostrar propiedades sobre todos los naturales. De manera similar, una relación inductiva como `aevalR` no sólo declara una estructura sintáctica, sino un sistema de reglas para construir derivaciones semánticas [3], [4], [9].

Otra diferencia importante es la disciplina sobre la recursión. En muchos lenguajes funcionales, se permite escribir funciones recursivas que podrían no terminar. Rocq restringe la recursión porque sus programas viven en el mismo marco que sus pruebas. Si se aceptaran definiciones no terminantes sin control, se pondría en riesgo la consistencia lógica del sistema. Por eso las funciones definidas con `Fixpoint` deben seguir una forma estructuralmente decreciente o justificarse mediante mecanismos aceptados por el sistema [5], [7]. Esta restricción no es un detalle accidental: es consecuencia de usar el mismo lenguaje para cómputo y demostración.

Esta comparación no implica que Rocq reemplace a los lenguajes funcionales de propósito general. Su objetivo principal no es competir como lenguaje de desarrollo cotidiano, sino proporcionar un entorno donde la programación funcional pueda integrarse con especificaciones y pruebas verificadas. La extracción de programas refuerza esta idea: Rocq puede generar código ejecutable a partir de definiciones verificadas, pero el valor principal del sistema está en que ese código puede provenir de un desarrollo formal donde la corrección fue establecida previamente [5], [12].

Por tanto, la diferencia central no está en la sintaxis funcional, sino en el significado formal de las construcciones. En Rocq, una función puede ser parte de una especificación, una relación inductiva puede representar un juicio semántico y un teorema puede certificar la conexión entre ambos. Esta integración es lo que permite interpretar la programación funcional en Rocq como un sistema lógico y no sólo como una forma de escribir programas.

VI. CONCLUSIONES

La programación funcional en Rocq puede interpretarse como un sistema lógico porque sus programas no son únicamente fragmentos de código ejecutable, sino términos tipados dentro de un marco formal basado en el Cálculo de Construcciones Inductivas. La respuesta a la pregunta de investigación se

concentra en un hecho técnico: Rocq verifica programas, proposiciones y pruebas mediante juicios de la misma forma,

$$\Gamma \vdash t : A. \quad (17)$$

Cuando A describe datos o funciones computacionales, t se interpreta como programa; cuando A expresa una proposición, t se interpreta como prueba. Por tanto, la relación entre programar y demostrar no es externa al sistema, sino parte de su organización formal.

La correspondencia de Curry–Howard proporciona la base conceptual de esta lectura. Bajo esta correspondencia, una proposición puede verse como un tipo y una prueba como un término que habita ese tipo. En Rocq, esta idea se materializa en construcciones concretas: las implicaciones se comportan como funciones, la cuantificación universal se expresa mediante productos dependientes y las proposiciones viven en `Prop`. Así, una demostración no es sólo una explicación informal, sino un objeto verificable por el sistema de tipos [1], [4], [6].

El papel de las definiciones inductivas refuerza esta interpretación. Con `Inductive`, Rocq permite definir no sólo estructuras de datos, sino también proposiciones, relaciones y juicios formales. Esto es esencial para ciencias de la computación, porque muchos conceptos semánticos se presentan precisamente como reglas de inferencia. Una relación como `aevalR` no es una función que calcula, sino una proposición que describe cuándo una expresión evalúa a un valor. Sus constructores son reglas para construir derivaciones, y esas derivaciones son objetos internos del sistema.

El caso de estudio mostró la conexión más importante del trabajo. La función `aeval` define una semántica computacional para expresiones aritméticas, mientras que la relación `aevalR` define una semántica lógica como juicio inductivo. El teorema

`Theorem aeval_iff_aevalR :`
`forall a n, aevalR a n <-> aeval a = n.`

certifica que ambas presentaciones coinciden. Este resultado resume la tesis del trabajo: en Rocq, un programa funcional puede relacionarse con una especificación lógica y con una prueba formal que verifica dicha relación dentro del mismo lenguaje.

Los *proof objects* explican por qué esta integración es posible. Una prueba escrita con tácticas no queda justificada por el script en sí mismo, sino por el término final que Rocq construye y verifica. Esto distingue a Rocq de una herramienta que sólo automatiza razonamientos: su núcleo comprueba que la prueba producida habite el tipo declarado. Por eso, una demostración en Rocq tiene el mismo estatus formal que otros términos del sistema, aunque su interpretación pertenezca al universo lógico.

Esta interpretación también permite precisar la diferencia con la programación funcional convencional. En lenguajes como OCaml, Haskell o ML, las funciones y los tipos permiten estructurar programas y evitar errores, pero las pruebas de corrección suelen permanecer fuera del lenguaje o depender de herramientas adicionales. En Rocq, en cambio, la función,

la especificación y la prueba pueden expresarse dentro del mismo sistema. La diferencia no es sólo sintáctica, sino lógica: los tipos pueden formular propiedades y las pruebas pueden construirse como términos.

Sin embargo, esta lectura debe entenderse con límites. No todo programa en Rocq constituye automáticamente una prueba matemática relevante. Una función de tipo `nat -> nat`, por sí sola, sólo clasifica una transformación entre datos. La lectura lógica se vuelve sustantiva cuando se formula una especificación adecuada y se construye una prueba de ella. Además, Rocq no elimina el trabajo conceptual de elegir la propiedad correcta: el sistema verifica que un término habite un tipo, pero el usuario debe formalizar correctamente qué quiere demostrar.

En conclusión, Rocq permite interpretar la programación funcional como sistema lógico porque unifica tres niveles que normalmente aparecen separados: código, especificación y demostración. Las funciones son términos; las proposiciones son tipos; las pruebas son términos que habitan esos tipos; y los juicios de evaluación o de corrección pueden formalizarse como relaciones inductivas. Por ello, programar en Rocq no consiste únicamente en escribir funciones que calculan, sino en construir objetos formales cuyo significado computacional y lógico puede ser verificado dentro del propio sistema.

REFERENCIAS

- [1] W. A. Howard, “The Formulae-as-Types Notion of Construction,” in *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, J. P. Seldin and J. R. Hindley, Eds. New York, NY, USA: Academic Press, 1980, pp. 479–490.
- [2] T. Coquand and G. Huet, “The Calculus of Constructions,” *Information and Computation*, vol. 76, no. 2–3, pp. 95–120, Feb./Mar. 1988.
- [3] C. Paulin-Mohring, “Introduction to the Calculus of Inductive Constructions,” in *All About Proofs, Proofs for All*, B. Woltzenlogel Paleo and D. Delahaye, Eds. London, U.K.: College Publications, 2015.
- [4] B. C. Pierce, A. A. de Amorim, C. Casinghino, M. Gaboardi, M. Greenberg, C. Hrițcu, V. Sjöberg, and B. Yorgey, *Software Foundations, Volume 1: Logical Foundations*. [Online]. Available: <https://softwarefoundations.cis.upenn.edu/lf-current/>
- [5] The Rocq Development Team, *The Rocq Prover Reference Manual*. [Online]. Available: <https://rocq-prover.org/refman>
- [6] B. C. Pierce et al., “Proof Objects,” in *Software Foundations, Volume 1: Logical Foundations*. [Online]. Available: <https://softwarefoundations.cis.upenn.edu/lf-current/ProofObjects.html>
- [7] A. Chlipala, *Certified Programming with Dependent Types*. Cambridge, MA, USA: MIT Press, 2013. [Online]. Available: <https://adam.chlipala.net/cpdt/>
- [8] Y. Bertot and P. Castéran, *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Berlin, Germany: Springer, 2004.
- [9] B. C. Pierce et al., “Inductively Defined Propositions,” in *Software Foundations, Volume 1: Logical Foundations*. [Online]. Available: <https://softwarefoundations.cis.upenn.edu/lf-current/IndProp.html>
- [10] B. C. Pierce, C. Casinghino, M. Greenberg, V. Sjöberg, and B. Yorgey, *Software Foundations, Volume 2: Programming Language Foundations*. [Online]. Available: <https://softwarefoundations.cis.upenn.edu/plf-current/>
- [11] B. C. Pierce et al., “Imp: Simple Imperative Programs,” in *Software Foundations, Volume 2: Programming Language Foundations*. [Online]. Available: <https://softwarefoundations.cis.upenn.edu/plf-current/Imp.html>
- [12] The Rocq Development Team, “Extraction of programs,” in *The Rocq Prover Reference Manual*. [Online]. Available: <https://rocq-prover.org/refman/addendum/extraction.html>